CTA-ED Exercise 4: Scaling techniques (with correct answers)

Zhitong Chen

10/03/2024

Introduction

The hands-on exercise for this week focuses on: 1) scaling texts ; 2) implementing scaling techniques using `quanteda`.

In this tutorial, you will learn how to:

- Scale texts using the “wordfish” algorithm
- Scale texts gathered from online sources
- Replicate analyses by @kaneko_estimating_2021

Before proceeding, we’ll load the packages we will need for this tutorial.

```
library(dplyr)
library(quanteda) # includes functions to implement Lexicoder
library(quanteda.textmodels) # for estimating similarity and complexity measures
library(quanteda.textplots) #for visualizing text modelling results
```

In this exercise we’ll be using the dataset we used for the sentiment analysis exercise. The data were collected from the Twitter accounts of the top eight newspapers in the UK by circulation. The tweets include any tweets by the news outlet from their main account.

Importing data

If you’re working on this document from your own computer (“locally”) you can download the tweets data in the following way:

```
tweets  <- readRDS(gzcon(url("https://github.com/cjbarrie/CTA-ED/blob/main/data/sentanalysis/newstweets.rds?raw=true")))
```

We first take a sample from these data to speed up the runtime of some of the analyses.

```
tweets <- tweets %>%
  sample_n(20000)
```

Construct `dfm` object

Then, as in the previous exercise, we create a corpus object, specify the document-level variables by which we want to group, and generate our document feature matrix.

```
#make corpus object, specifying tweet as text field
tweets_corpus <- corpus(tweets, text_field = "text")

#add in username document-level information
docvars(tweets_corpus, "newspaper") <- tweets$user_username

dfm_tweets <- dfm(tokens(tweets_corpus,
                    remove_punct = TRUE)) %>%
  dfm_select(pattern = stopwords("english"),
             selection = "remove",
             valuetype = "fixed")
```

We can then have a look at the number of documents (tweets) we have per newspaper Twitter account.

```
## number of tweets per newspaper
table(docvars(dfm_tweets, "newspaper"))
```

```
##
##     DailyMailUK     DailyMirror EveningStandard        guardian         MetroUK
##            2058            5766            2134            2874             941
##       Telegraph          TheSun        thetimes
##            1554            3982             691
```

And this is what our document feature matrix looks like, where each word has a count for each of our eight newspapers.

```
dfm_tweets
```

```
## Document-feature matrix of: 20,000 documents, 49,324 features (99.98% sparse) and 31 docvar
s.
##        features
## docs    two dead deadly new pneumonia virus strain outbreak spreading china
##   text1   1    1      1   1         1     1      1        1         1     1
##   text2   0    0      0   0         0     0      0        0         0     0
##   text3   0    0      0   0         0     0      0        0         0     0
##   text4   0    0      0   0         0     0      0        0         0     0
##   text5   0    0      0   0         0     0      0        0         0     0
##   text6   0    0      0   0         0     0      0        0         0     0
## [ reached max_ndoc ... 19,994 more documents, reached max_nfeat ... 49,314 more features ]
```

Estimate wordfish model

Once we have our data in this format, we are able to group and trim the document feature matrix before estimating the wordfish model.

```
# compress the document-feature matrix at the newspaper level
dfm_newstweets <- dfm_group(dfm_tweets, groups = newspaper)

# remove words not used by two or more newspapers
dfm_newstweets <- dfm_trim(dfm_newstweets,
                               min_docfreq = 2, docfreq_type = "count")

## size of the document-feature matrix
dim(dfm_newstweets)
```

```
## [1]     8 11229
```

```
#### estimate the Wordfish model ####
set.seed(123L)
dfm_newstweets_results <- textmodel_wordfish(dfm_newstweets,
                                             )
```

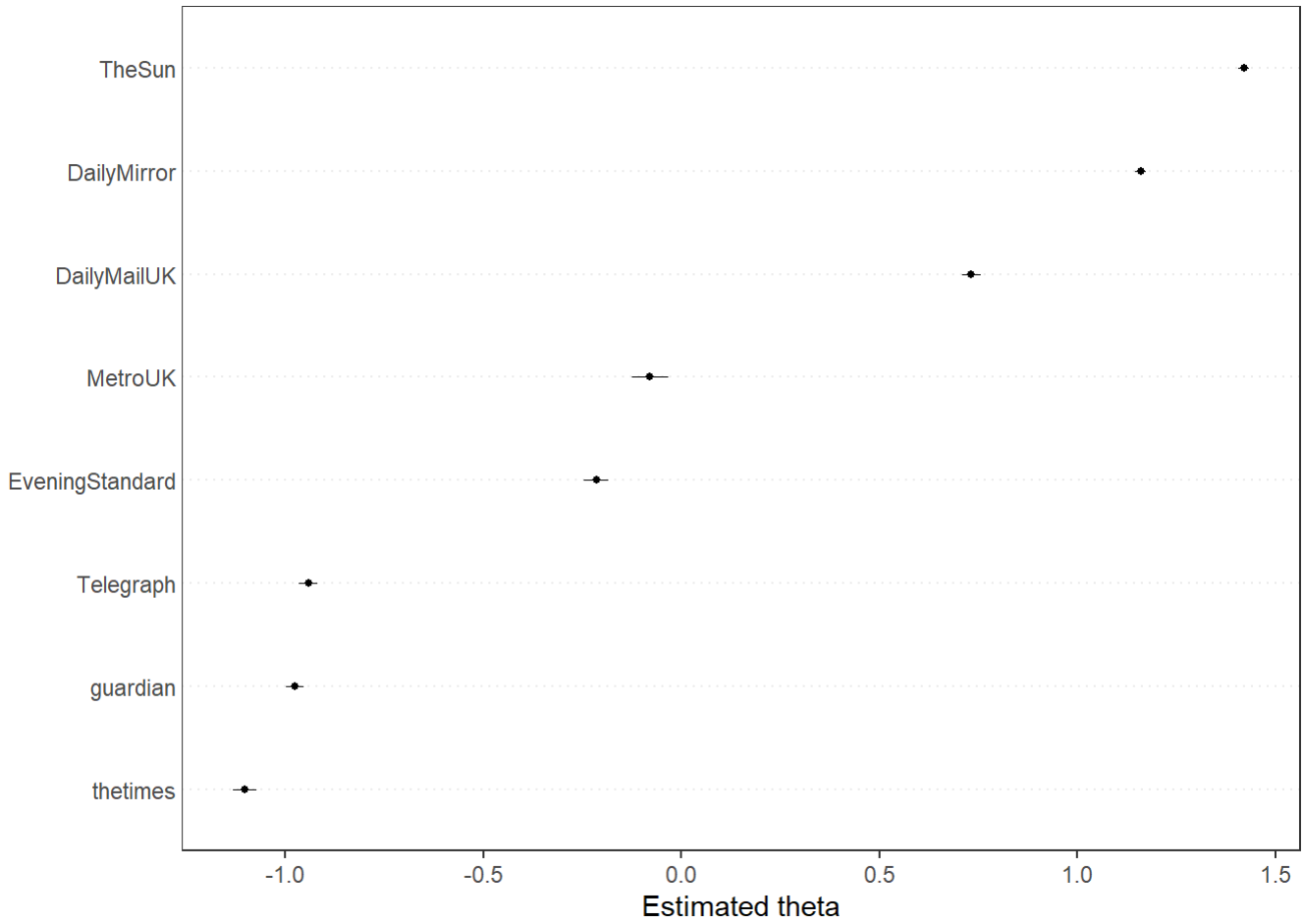
And this is what results.

```
summary(dfm_newstweets_results)
```

```
##
## Call:
## textmodel_wordfish.dfm(x = dfm_newstweets)
##
## Estimated Document Positions:
##                    theta       se
## DailyMailUK      0.73226 0.012317
## DailyMirror      1.15959 0.006699
## EveningStandard -0.21410 0.015959
## guardian        -0.97535 0.011022
## MetroUK         -0.07932 0.023210
## Telegraph       -0.94104 0.012195
## TheSun           1.41985 0.006861
## thetimes        -1.10190 0.015371
##
## Estimated Feature Scores:
##         two   dead deadly     new pneumonia   virus   strain outbreak spreading
## beta 0.1111 0.2972 0.3997 -0.1746    0.6427 -0.4458  0.35534  -0.1453   -0.1445
## psi  3.6027 2.8551 1.7639  4.5804   -0.6021  3.0945 -0.05475   3.1187    0.9358
##        china  oasis definitely    get    back  together insists secret woman
## beta -0.4627  1.754      3.532 0.1375 -0.0121 -0.006017  0.3382 0.5241 0.721
## psi   2.9479 -2.767     -3.886 3.6470  3.5196  2.316239  1.2482 1.9298 3.188
##       behind success   girl shocked    find     3ft   snake lurking family's
## beta 0.03445  0.1134 0.9571  0.4951 -0.1865  0.6998  0.7677   1.121  0.72222
## psi  2.50510  1.0635 2.3330  0.7381  2.7093 -2.0394 -0.5980  -2.051 -0.04499
##      bathroom scottish     cup
## beta   1.7195  -0.4729 -0.6214
## psi   -0.8292   0.9328  1.7970
```

We can then plot our estimates of the θs—i.e., the estimates of the latent newspaper position—as so.

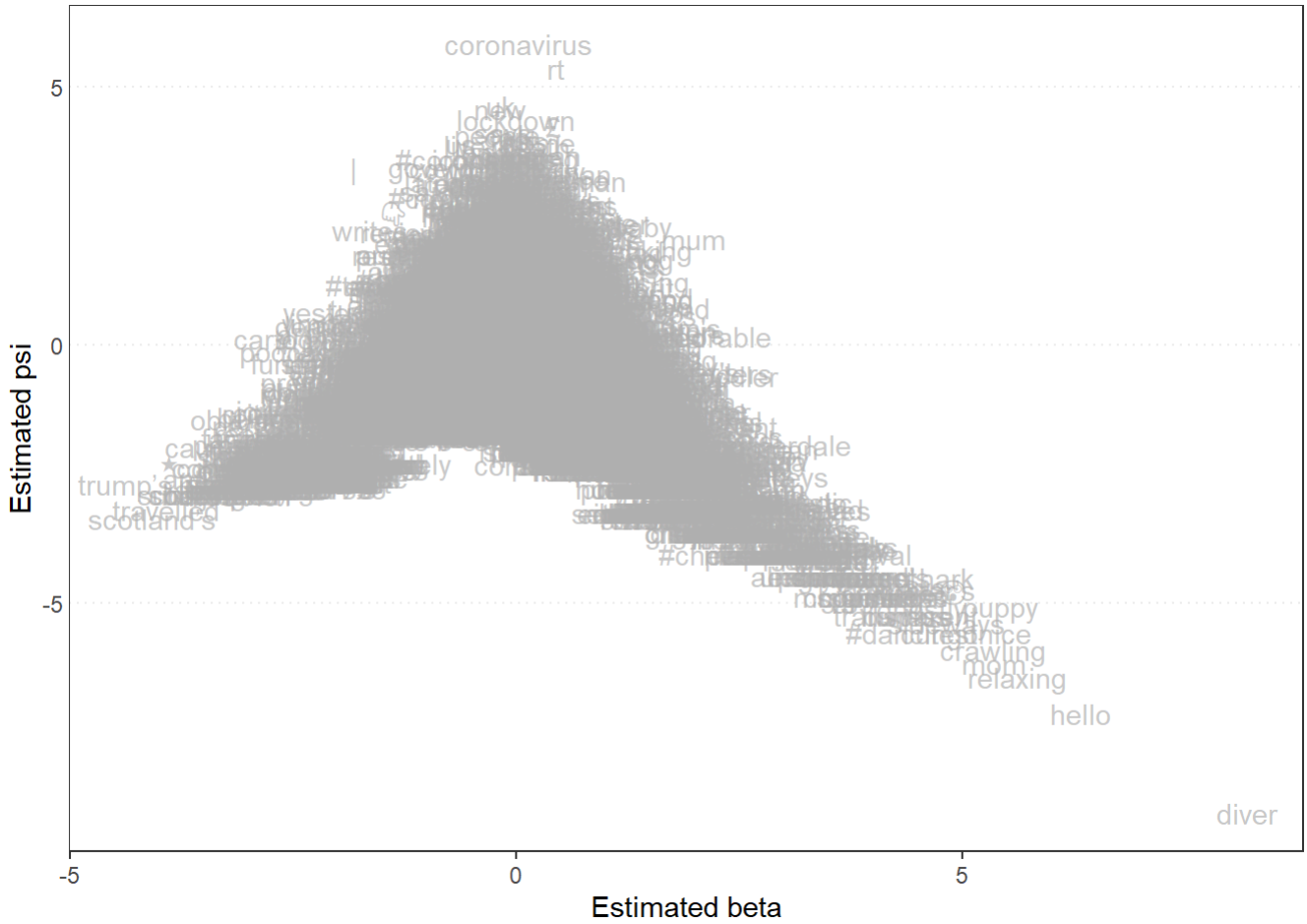
```
textplot_scaleld(dfm_newstweets_results)
```



Interestingly, we seem not to have captured ideology but some other tonal dimension. We see that the tabloid newspapers are scored similarly, and grouped toward the right hand side of this latent dimension; whereas the broadsheet newspapers have an estimated theta further to the left.

Plotting the “features,” i.e., the word-level betas shows how words are positioned along this dimension, and which words help discriminate between news outlets.

```
textplot_scaleld(dfm_newstweets_results, margin = "features")
```



And we can also look at these features.

```
features <- dfm_newstweets_results[["features"]]

betas <- dfm_newstweets_results[["beta"]]

feat_betas <- as.data.frame(cbind(features, betas))
feat_betas$betas <- as.numeric(feat_betas$betas)

feat_betas %>%
  arrange(desc(betas)) %>%
  top_n(20) %>%
  kbl() %>%
  kable_styling(bootstrap_options = "striped")
```

```
## Selecting by betas
```

features	betas
diver	8.188060
hello	6.320093
relaxing	5.619455
puppy	5.430632
mom	5.348476

features	betas
crawling	5.337777
😱	4.892488
sideways	4.827735
shark	4.746262
#dancingonice	4.730202
cutest	4.730202
ghostly	4.542586
toys	4.525696
daughter's	4.403968
transparent	4.373034
dumps	4.373034
lioness	4.373034
wwe	4.370189
cristiano	4.233490
girl's	3.975124
mcguinness	3.975124
chopsticks	3.975124
spinning	3.975124
giggling	3.975124
airs	3.975124
sacred	3.975124

These words do seem to belong to more tabloid-style reportage, and include emojis relating to film, sports reporting on "cristiano" as well as more colloquial terms like "saucy."

Replicating Kaneko et al.

This section adapts code from the replication data provided for @kaneko_estimating_2021 here (https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/EL3KYD).

If you're working locally, you can download the `dfm` data with:

```
kaneko_dfm  <- readRDS(gzcon(url("https://github.com/cjbarrie/CTA-ED/blob/main/data/wordscaling/study1_kaneko.rds?raw=true")))
```

This data is in the form a document-feature-matrix. We can first manipulate it in the same way as @kaneko_estimating_2021 by grouping at the level of newspaper and removing infrequent words.

```
table(docvars(kaneko_dfm, "Newspaper"))
```

```
##
##       Asahi     Chugoku    Chunichi    Hokkaido      Kahoku    Mainichi
##          38          24          47          46          18          26
##      Nikkei Nishinippon      Sankei     Yomiuri
##          13          27          14          30
```

```
## prepare the newspaper-level document-feature matrix
# compress the document-feature matrix at the newspaper level
kaneko_dfm_study1 <- dfm_group(kaneko_dfm, groups = Newspaper)
# remove words not used by two or more newspapers
kaneko_dfm_study1 <- dfm_trim(kaneko_dfm_study1, min_docfreq = 2, docfreq_type = "count")

## size of the document-feature matrix
dim(kaneko_dfm_study1)
```

```
## [1]   10 4660
```

Exercises

1. Estimate a wordfish model for with Kaneko (2021)'s data

```
## estimate the Wordfish model

# Set a random seed to ensure reproducibility of the model
set.seed(123L)

# Fit the Wordfish model
kaneko_results <- textmodel_wordfish(kaneko_dfm_study1)

# Print a summary of the fitted model
summary(kaneko_results)
```

```
##
## Call:
## textmodel_wordfish.dfm(x = kaneko_dfm_study1)
##
## Estimated Document Positions:
##               theta      se
## Asahi       -0.9892 0.01651
## Chugoku     -0.4585 0.02446
## Chunichi    -1.2230 0.01232
## Hokkaido    -0.4160 0.01917
## Kahoku      -0.5731 0.02753
## Mainichi    -0.1876 0.02546
## Nikkei       1.2578 0.04403
## Nishinippon -0.2616 0.02750
## Sankei       1.2472 0.03943
## Yomiuri      1.6040 0.02477
##
## Estimated Feature Scores:
##        安保     法制     国会     一線    越え 安倍内閣   新た 安全保障政策
## beta 0.2157 0.01591 0.09476 -0.05989 -1.362  -1.6217 0.332    -0.006181
## psi  3.6911 3.36064 3.73441 -0.18786 -1.989   0.4758 2.192     0.789972
##      関連法案    閣議    決定    提出 安倍首相    先月     米    議会     演説
## beta  0.01405 -0.1579 -0.2177 0.5569   0.2524 -0.6656 0.5448 -0.9467 -0.22245
## psi   2.41503  0.7228  1.2414 2.5076   2.5758 -0.5345 2.8280  0.5425 -0.08278
##      安全保障    戦後   初めて 大改革      夏   成就    約束    通り    合意
## beta   0.2303 -0.3609 0.003823 -1.362 -0.6648 -2.335 -0.6655 -0.4473 0.4545
## psi    3.0372  2.0478 0.937039 -1.989  0.6898 -2.481  0.1587  0.4404 1.8744
##      歴史的  転換 集団的自衛権     行使
## beta -1.154 0.141       0.2183 0.05474
## psi  -2.105 1.609       3.7932 3.73714
```

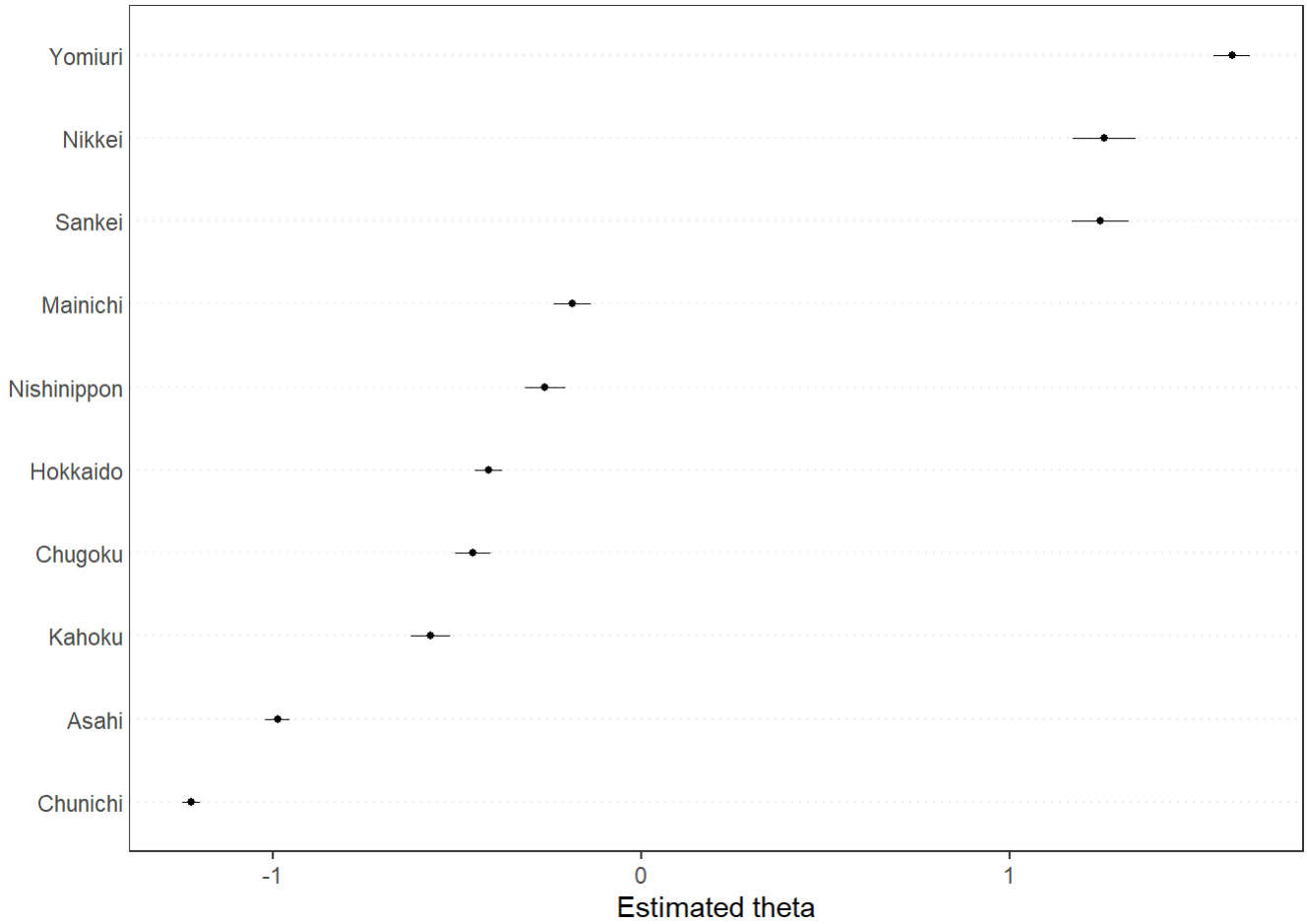
Write a paragraph here explaining and interpreting your results

The Wordfish model estimates each newspaper's latent ideological position (theta, θ) on a single underlying dimension, inferred purely from word-usage patterns — without any pre-assigned labels. The results reveal a clear left–right spectrum among the ten Japanese newspapers in their coverage of the 2015 security legislation (安保法制). On the liberal/left end, Chunichi (θ = −1.22), Asahi (θ = −0.99), and Kahoku (θ = −0.57) score most negatively, suggesting they use language more critical of the legislation. On the conservative/right end, Yomiuri (θ = 1.60), Nikkei (θ = 1.26), and Sankei (θ = 1.25) score most positively, indicating language more supportive or neutral toward the Abe cabinet's agenda. The feature scores (beta, β) reinforce this interpretation: words like 安倍内閣 (β = −1.62) and 越え (β = −1.36, implying "crossing a line/overstepping") are associated with the liberal pole, while terms like 米 ("U.S.", β = +0.54) and 安保 ("security alliance", β = +0.22) lean toward the conservative pole. The small standard errors across all newspapers indicate high estimation precision, lending confidence to these positional estimates.

2. Visualize the results

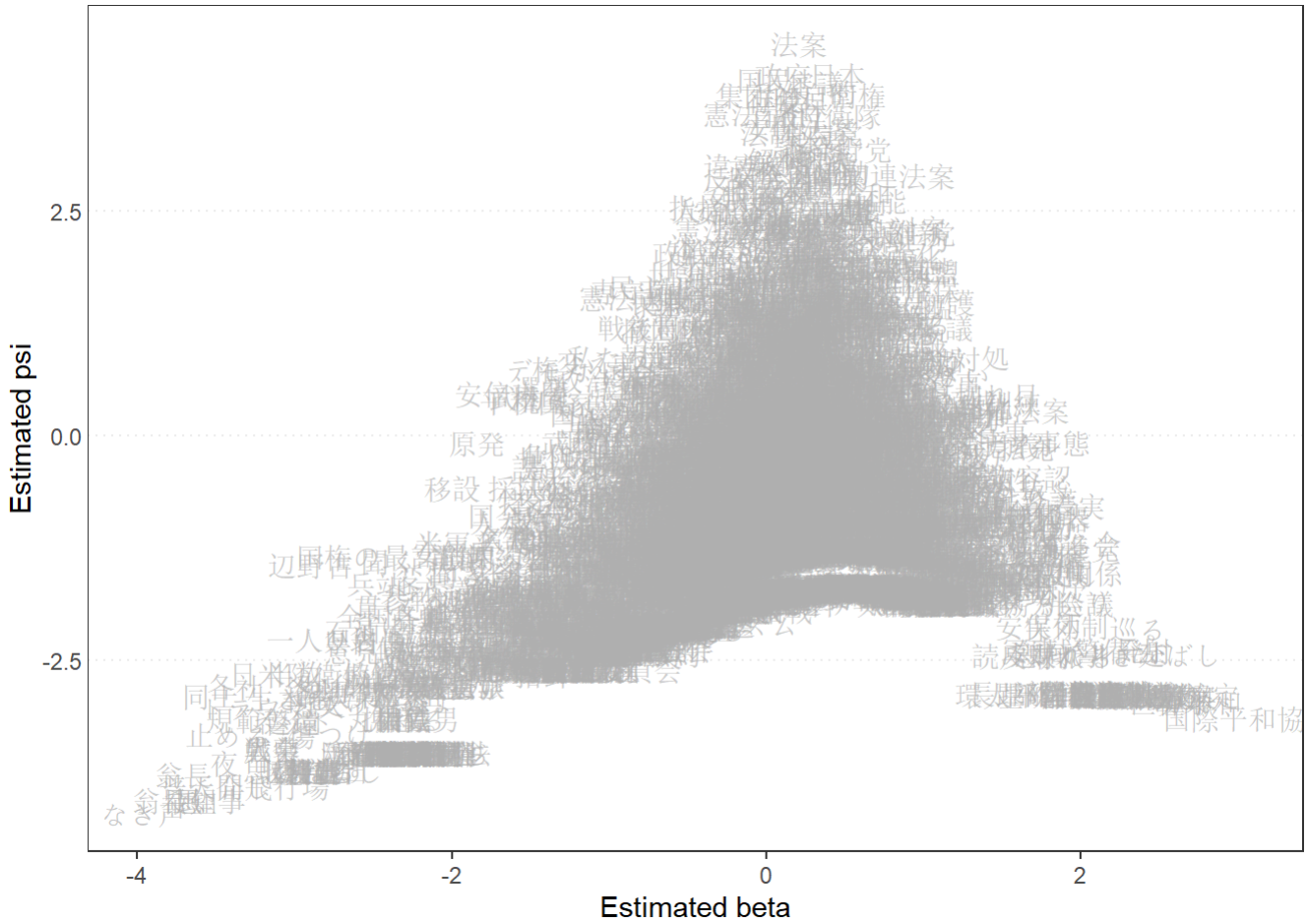
```
## We can then plot our estimates of the thetas---i.e., the estimates of the latent Japanese ne
wspaper position.

# Plot 1: Document-level scaling (theta estimates per newspaper) # textplot_scaleld() visualize
s the estimated latent positions (thetas) with 95% confidence intervals shown as horizontal bar
s
textplot_scaleld(kaneko_results)
```



Wordfish model estimates of Japanese newspapers' editorial texts

```
# Plot 2: Feature-level scaling (beta estimates per word)
# margin = "features" switches the plot to show word-level betas
textplot_scaleld(kaneko_results, margin = "features")
```



Wordfish model estimates of Japanese newspapers' editorial texts

```
# Optional: Inspect top discriminating words manually ---

# Extract the feature (word) names from the model
features_k <- kaneko_results[["features"]]

# Extract the beta coefficients (word-level discrimination parameters)
betas_k <- kaneko_results[["beta"]]

# Combine into a data frame for easy sorting and inspection
feat_betas_k <- as.data.frame(cbind(features_k, betas_k))

# Convert beta column from character to numeric for correct sorting
feat_betas_k$betas_k <- as.numeric(feat_betas_k$betas_k)

# Sort by descending beta and show top 20 most discriminating words
feat_betas_k %>%
  arrange(desc(betas_k)) %>%   # Sort from highest to lowest beta
  top_n(20) %>%                # Keep only the top 20 rows
  kbl() %>%                    # Format as a nice table
  kable_styling(bootstrap_options = "striped")  # Add striped styling
```

```
## Selecting by betas_k
```

features_k	betas_k
国際平和協力	3.065431

features_k	betas_k
極小	2.665645
世界平和	2.665645
勘案	2.649405
引き延ばし	2.453737
心がけ	2.418021
気がかり	2.418021
乱れ	2.418021
享受	2.418021
中国軍	2.418021
世界秩序	2.418021
取り組め	2.418021
問責決議案	2.418021
農協	2.403045
横行	2.403045
収める	2.403045
除き	2.403045
巡る	2.365867
合戦	2.109551
合区	2.109551
演出	2.109551
具体策	2.109551
騒動	2.109551
離合集散	2.109551
遅滞	2.109551
変調	2.109551
環太平洋経済連携協定	2.109551
活性	2.109551

features_k	betas_k
消費税率	2.109551
基礎的財政収支	2.109551
黒字	2.109551
佐藤正久	2.109551

Write here the interpretation of your plot(s) Plot 1: Document-Level Scaling (Theta Estimates) The dot plot displays each newspaper's estimated ideological position (θ) with 95% confidence intervals. The horizontal spread confirms a robust left–right divide: Chunichi and Asahi anchor the liberal pole (θ ≈ −1.22, −0.99), while Yomiuri, Nikkei, and Sankei occupy the conservative pole (θ ≈ 1.60, 1.26, 1.25). The middle cluster — Mainichi, Nishinippon, Hokkaido, Chugoku, and Kahoku — sits in mildly negative territory, suggesting a centre-left lean. Critically, the confidence intervals are very narrow and non-overlapping between the two poles, indicating that the model's positional estimates are statistically well-separated and reliable.

Plot 2: Feature-Level Scaling (Beta–Psi Scatter) This plot maps every word in the vocabulary by its beta (x-axis: ideological discrimination) and psi (y-axis: overall word frequency). Words near β = 0 are used equally across all newspapers and thus carry little discriminating information. Words with strongly negative beta lean toward the liberal pole; words with strongly positive beta lean conservative. The dense central mass reflects the many common, uninformative words. The sparse tails — especially words with β < −3 or β > +2 — are the ideologically "loaded" terms that most distinguish conservative from liberal coverage.

Top Discriminating Words (Highest Beta) The top-beta words reveal what language is disproportionately used by conservative-leaning papers. 国際平和協力 ("international peace cooperation", β = 3.07) and 世界平和 ("world peace", β = 2.67) are striking — conservative papers appear to frame the security legislation in the language of positive international contribution, whereas liberal papers likely use more adversarial or constitutional-crisis framing. Words like 引き延ばし ("delay/stalling", β = 2.45) and 気がかり ("concern/worry", β = 2.42) suggest conservative papers also reported on legislative process friction and public anxiety, but from a pro-passage perspective.

Don't forget to knit!